

Unidad 8: Manejo de Archivos y Datos en Python

Introducción al Manejo de Archivos

Persistencia

La persistencia en programación se refiere a la capacidad de almacenar y recuperar datos de manera permanente, de tal forma que estos datos sobrevivan al cierre de la aplicación que los creó. Esto permite que la información sea reutilizable en futuras ejecuciones del programa.

Datos Efímeros vs. Datos Persistentes

- **Datos Efímeros:** Son aquellos datos que solo existen mientras el programa está en ejecución. Una vez que el programa se cierra, estos datos se pierden. Un ejemplo de datos efímeros podría ser una variable en memoria que se utiliza para cálculos temporales.
 - **Datos Persistentes:** Son aquellos que se almacenan de manera permanente, de modo que pueden ser recuperados y utilizados en futuras ejecuciones del programa. Estos datos se guardan en medios de almacenamiento no volátiles, como discos duros o SSDs.
-

Almacenamiento en Archivos y Bases de Datos

Para lograr la persistencia de datos, se utilizan dos métodos principales: el almacenamiento en archivos y el almacenamiento en bases de datos.

Archivos

Los datos se almacenan en archivos en el sistema de archivos del ordenador. Los archivos pueden ser de diferentes tipos, como texto, binarios, CSV, JSON, entre otros. Trabajar con archivos es generalmente sencillo y adecuado para aplicaciones pequeñas o cuando se manejan datos de manera simple.

Bases de Datos

Los datos se almacenan en sistemas de gestión de bases de datos (DBMS), que permiten el manejo eficiente de grandes volúmenes de datos, la ejecución de consultas complejas y la integridad de los datos. Las bases de datos son ideales para aplicaciones grandes y complejas donde la organización y acceso a los datos son cruciales.

Tipos de Persistencia

Archivos

El almacenamiento en archivos es una manera directa y sencilla de lograr la persistencia. Los archivos pueden ser manipulados utilizando funciones de lectura y escritura que permiten guardar y recuperar datos según sea necesario.

Ventajas:

- Simplicidad: Fácil de implementar y entender.
- Portabilidad: Los archivos pueden ser fácilmente trasladados entre sistemas.

Desventajas:

- Escalabilidad: No es ideal para manejar grandes volúmenes de datos.
 - Integridad: Más propenso a errores y pérdida de datos si no se maneja adecuadamente.
-

Tipos de Persistencia

Bases de Datos

El almacenamiento en bases de datos es una solución más robusta y escalable para la persistencia de datos. Utiliza un sistema de gestión de bases de datos (DBMS) para manejar la creación, consulta, actualización y eliminación de datos de manera eficiente.

Ventajas:

- Escalabilidad: Maneja grandes volúmenes de datos de manera eficiente.
- Integridad y Consistencia: Ofrece mecanismos para asegurar que los datos sean precisos y consistentes.
- Consultas Complejas: Permite la ejecución de consultas avanzadas para obtener información específica.

Desventajas:

- Complejidad: Requiere conocimientos más avanzados para implementar y mantener.
 - Costo: Puede implicar costos adicionales en términos de licencias y mantenimiento.
-

En esta unidad, nos enfocaremos principalmente en el almacenamiento en archivos. Aprenderemos a leer y escribir datos en archivos de texto y binarios, y cómo utilizar formatos comunes como CSV y JSON para estructurar nuestros datos.

En unidades posteriores, exploraremos en detalle el uso de bases de datos, incluyendo cómo diseñarlas, implementarlas y usarlas para manejar datos de manera eficiente.

Ejemplo de Uso de Archivos

Imaginemos que tenemos una aplicación que necesita guardar los resultados de una encuesta. Podríamos almacenar estos resultados en un archivo CSV, donde cada línea del archivo representa una respuesta de la encuesta.

Ejemplo de archivo CSV:

...

```
nombre,edad,opinión
Juan,28,Me gusta
Ana,34,No me gusta
Pedro,23,Me encanta
```

...

Podemos leer y escribir este archivo utilizando funciones en Python, permitiendo que los datos sean persistentes y accesibles en futuras ejecuciones de nuestra aplicación.

Con estos conceptos básicos, estarás preparado para manejar la persistencia de datos en tus aplicaciones, empezando con archivos y avanzando eventualmente hacia el uso de bases de datos en unidades posteriores.

Archivos

¿Qué son los archivos?

Un archivo informático es una secuencia de bytes almacenados en un dispositivo de almacenamiento, como un disco duro, SSD o unidad USB. Cada archivo está identificado por un nombre y su ubicación en un directorio o carpeta. Los archivos pueden contener diferentes tipos de datos, desde texto simple hasta datos binarios complejos.

Tipos de Archivos

Archivos de Texto

Los archivos de texto contienen datos en un formato legible por humanos, utilizando caracteres de un conjunto de caracteres como ASCII o UTF-8. Cada línea de texto está separada por un carácter de nueva línea.

Ejemplo:

...

Hola, mundo
Este es un archivo de texto

...

Archivos Binarios

Los archivos binarios contienen datos en un formato no legible por humanos. Estos archivos pueden incluir imágenes, vídeos, programas ejecutables y cualquier otro tipo de datos que no sean texto.

Ejemplo: Un archivo de imagen `.jpg`, un archivo de vídeo `.mp4`.

Enfoque en Archivos de Texto

Para este curso, nos centraremos principalmente en archivos de texto. Esto incluye cómo leer y escribir datos en estos archivos usando Python, ya que son más sencillos de manejar y entender para principiantes.

Lectura de Archivos

Python proporciona varias formas de leer datos de un archivo de texto. A continuación, describimos las tres formas principales de hacerlo: `read()`, `readline()`, y `readlines()`.

Método `read()`

El método `read()` lee todo el contenido del archivo y lo devuelve como una cadena de texto. Es útil cuando necesitas trabajar con todo el contenido del archivo de una sola vez.

Ejemplo:

```
```python
with open('archivo.txt', 'r') as archivo:
 contenido = archivo.read()
 print(contenido)
...

```

---

## Método `readline()`

El método `readline()` lee una sola línea del archivo a la vez. Es útil cuando quieres procesar el archivo línea por línea.

**Ejemplo:**

```
```python
with open('archivo.txt', 'r') as archivo:
    linea = archivo.readline()
    while linea:
        print(linea, end=") # El end=" evita que se agregue una nueva línea adicional
        linea = archivo.readline()
...

```

Método `readlines()`

El método `readlines()` lee todas las líneas del archivo y las devuelve como una lista de cadenas. Cada elemento de la lista representa una línea del archivo.

Ejemplo:

```
```python
with open('archivo.txt', 'r') as archivo:

```

```
lineas = archivo.readlines()
for linea in lineas:
 print(linea, end="")
```

...

---

## Resumen de los Métodos

- ``read()``: Lee todo el archivo como una única cadena.
- ``readline()``: Lee una línea del archivo a la vez.
- ``readlines()``: Lee todas las líneas del archivo y las devuelve como una lista de cadenas.

Estos métodos proporcionan flexibilidad para manejar archivos de texto en Python, permitiendo leer desde archivos pequeños hasta archivos de gran tamaño de manera eficiente.

---

## Trabajando con Archivos JSON

### Introducción a los Archivos JSON

JSON (JavaScript Object Notation) es un formato de texto ligero para el intercambio de datos. Es fácil de leer y escribir para los humanos, y fácil de interpretar y generar para las máquinas. JSON es ampliamente utilizado para el almacenamiento y la transmisión de datos estructurados, especialmente en aplicaciones web y APIs.



# ¿Qué es JSON?

JSON representa datos como pares clave-valor. Las estructuras de datos comunes en JSON incluyen objetos (representados como diccionarios en Python) y arrays (representados como listas en Python).

## Ejemplo de un objeto JSON:

```
```json
{
  "nombre": "Juan",
  "edad": 30,
  "ciudad": "Madrid"
}
```
```

## Ejemplo de un array JSON:

```
```json
[
  {
    "nombre": "Juan",
    "edad": 30,
    "ciudad": "Madrid"
  },
  {
    "nombre": "Ana",
    "edad": 25,
    "ciudad": "Barcelona"
  }
]
```
```

---

# Utilidad de JSON

JSON es útil para guardar datos estructurados de una manera que sea fácil de acceder y manipular. Se utiliza comúnmente para la configuración de aplicaciones, intercambio de datos entre un cliente y un servidor, y almacenamiento de datos en archivos.

## Trabajando con JSON en Python

Python proporciona una biblioteca estándar llamada `json` para trabajar con datos JSON. Esta biblioteca permite la serialización (convertir un objeto de Python en una cadena JSON) y la deserialización (convertir una cadena JSON en un objeto de Python).

---

### Escribir Archivos JSON (Serialización)

Para escribir datos en un archivo JSON, se utiliza el método `json.dump()`.

#### Ejemplo:

```
```python

import json

# Datos a serializar
datos = {
    "nombre": "Juan",
    "edad": 30,
    "ciudad": "Madrid"
}

# Serialización y escritura en un archivo JSON
with open('datos.json', 'w') as archivo:
    json.dump(datos, archivo)

...
```
```

---

### Leer Archivos JSON (Deserialización)

Para leer datos desde un archivo JSON, se utiliza el método `json.load()`.

#### Ejemplo:

```
```python

import json

# Lectura y deserialización desde un archivo JSON
with open('datos.json', 'r') as archivo:
    datos = json.load(archivo)

print(datos)
```
```

```
'''
```

---

## Serialización y Deserialización de Cadenas JSON

Además de trabajar con archivos, también puedes serializar y deserializar cadenas JSON utilizando `json.dumps()` y `json.loads()`.

### Ejemplo de Serialización:

```
'''python

import json

Datos a serializar
datos = {
 "nombre": "Ana",
 "edad": 25,
 "ciudad": "Barcelona"
}

Serialización a una cadena JSON
cadena_json = json.dumps(datos)
print(cadena_json)

'''
```

### Ejemplo de Deserialización:

```
'''python

import json

Cadena JSON a deserializar
cadena_json = '{"nombre": "Ana", "edad": 25, "ciudad": "Barcelona"}'

Deserialización a un objeto de Python
datos = json.loads(cadena_json)
print(datos)

'''
```

---

# Resumen

- **JSON:** Un formato de texto ligero para el intercambio de datos.
- **Serialización:** Convertir un objeto de Python en una cadena JSON (uso de ``json.dump()`` para archivos y ``json.dumps()`` para cadenas).
- **Deserialización:** Convertir una cadena JSON en un objeto de Python (uso de ``json.load()`` para archivos y ``json.loads()`` para cadenas).

Trabajar con JSON en Python es una habilidad fundamental para manejar datos estructurados de manera eficiente y efectiva. Con estos conceptos y ejemplos, estarás preparado para utilizar archivos JSON en tus aplicaciones.

---

## Trabajo con Datos Reales

### Archivos CSV

Un archivo CSV (Comma-Separated Values) es un formato de archivo utilizado para almacenar datos tabulares en texto plano. Cada línea del archivo representa un registro de datos, y los valores de cada registro están separados por comas (u otros delimitadores como punto y coma o tabulaciones). Los archivos CSV son ampliamente utilizados debido a su simplicidad y compatibilidad con muchas aplicaciones y herramientas de análisis de datos.



---

# Leer y Escribir Archivos CSV en Python

Python proporciona la biblioteca estándar `csv` para trabajar con archivos CSV. Esta biblioteca permite leer y escribir archivos CSV de manera eficiente.

## Leer Archivos CSV

Para leer datos desde un archivo CSV, se utiliza la función `csv.reader()`. Este método devuelve un objeto que itera sobre las líneas del archivo CSV.

### Ejemplo:

```
```python

import csv

with open('datos.csv', newline='') as archivo_csv:
    lector_csv = csv.reader(archivo_csv)
    for fila in lector_csv:
        print(fila)

```
```

---

## Escribir Archivos CSV

Para escribir datos en un archivo CSV, se utiliza la función `csv.writer()`. Este método proporciona un objeto que tiene métodos para escribir datos en el archivo CSV.

### Ejemplo:

```
```python

import csv

# Datos a escribir en el archivo CSV
datos = [
    ['nombre', 'edad', 'ciudad'],
    ['Juan', '30', 'Madrid'],
    ['Ana', '25', 'Barcelona'],
    ['Pedro', '23', 'Sevilla']
]

with open('salida.csv', 'w', newline='') as archivo_csv:
    escritor_csv = csv.writer(archivo_csv)
    escritor_csv.writerows(datos)

```
```

```
'''
```

---

## Manipulación de Datos CSV

La biblioteca `csv` también permite trabajar con archivos CSV utilizando diccionarios, lo que puede ser más intuitivo para algunos casos.

### Ejemplo de Lectura con Diccionarios:

```
'''python

import csv

with open('datos.csv', newline='') as archivo_csv:
 lector_dict = csv.DictReader(archivo_csv)
 for fila in lector_dict:
 print(fila)

'''
```

### Ejemplo de Escritura con Diccionarios:

```
'''python

import csv

Datos a escribir en el archivo CSV
datos = [
 {'nombre': 'Juan', 'edad': 30, 'ciudad': 'Madrid'},
 {'nombre': 'Ana', 'edad': 25, 'ciudad': 'Barcelona'},
 {'nombre': 'Pedro', 'edad': 23, 'ciudad': 'Sevilla'}
]

campos = ['nombre', 'edad', 'ciudad']

with open('salida.csv', 'w', newline='') as archivo_csv:
 escritor_dict = csv.DictWriter(archivo_csv, fieldnames=campos)
 escritor_dict.writeheader()
 escritor_dict.writerows(datos)

'''
```

---

## Ejemplo de Trabajo con Archivos CSV

A continuación, se presenta un ejemplo práctico de cómo leer y escribir archivos CSV en Python. Este ejemplo muestra cómo abrir un archivo CSV, leer sus datos y escribir nuevos datos en un archivo CSV.

### Ejemplo Detallado

#### Paso 1: Leer Datos desde un Archivo CSV

Primero, creamos un archivo CSV de ejemplo llamado `estudiantes.csv` con el siguiente contenido:

```
...

nombre,edad,grado
Juan,15,10
Ana,14,9
Pedro,16,11

...
```

Luego, utilizamos Python para leer los datos de este archivo.

```
```python  
  
import csv  
  
# Leer datos desde el archivo CSV  
with open('estudiantes.csv', newline='') as archivo_csv:  
    lector_csv = csv.reader(archivo_csv)  
    # Leer la primera línea (encabezados), esto hace que  
    # el siguiente bucle inicie luego del encabezado  
    encabezados = next(lector_csv)  
    for fila in lector_csv:  
        print(fila)  
  
...
```

Paso 2: Procesar los Datos

Podemos procesar los datos leídos para realizar alguna operación, como calcular la edad promedio de los estudiantes.

```
```python  

import csv
```

```
total_edad = 0
conteo_estudiantes = 0

with open('estudiantes.csv', newline='') as archivo_csv:
 lector_csv = csv.reader(archivo_csv)
 encabezados = next(lector_csv) # Leer la primera línea (encabezados)
 for fila in lector_csv:
 edad = int(fila[1])
 total_edad += edad
 conteo_estudiantes += 1

edad_promedio = total_edad / conteo_estudiantes
print(f'Edad promedio: {edad_promedio}')

...

```

---

### Paso 3: Escribir Nuevos Datos en un Archivo CSV

Finalmente, escribimos los datos procesados en un nuevo archivo CSV llamado `resumen\_estudiantes.csv`.

```
```python

import csv

# Datos a escribir en el archivo CSV
datos_resumen = [
    ['nombre', 'edad', 'grado'],
    ['Juan', '15', '10'],
    ['Ana', '14', '9'],
    ['Pedro', '16', '11'],
    ['Edad promedio', edad_promedio, ""]
]

with open('resumen_estudiantes.csv', 'w', newline='') as archivo_csv:
    escritor_csv = csv.writer(archivo_csv)
    escritor_csv.writerows(datos_resumen)

...

```

Conclusión

Trabajar con archivos CSV en Python es sencillo y eficaz utilizando la biblioteca `csv`. Este ejemplo práctico demuestra cómo leer, procesar y escribir datos en archivos CSV,

proporcionando una base sólida para manejar datos tabulares en tus proyectos.

Uso de la Función `with`

En Python, la función `with` se utiliza para manejar archivos de manera segura y eficiente. Esta función asegura que los archivos se abran y cierren correctamente, incluso si ocurre una excepción durante el proceso de lectura o escritura. El uso de `with` simplifica el código y reduce el riesgo de errores relacionados con el manejo de archivos.

¿Qué es la Función `with`?

La función `with` en Python es un gestor de contexto que facilita la gestión de recursos como archivos, conexiones de red, o incluso bloques de código que deben garantizar la liberación de recursos al finalizar su uso. Cuando se utiliza `with` para abrir un archivo, se asegura que el archivo se cierre automáticamente cuando se sale del bloque de código, independientemente de cómo se salga (ya sea de manera normal o debido a una excepción).

Sintaxis de la Función `with`

La sintaxis básica para utilizar `with` al abrir un archivo es la siguiente:

```
```python
with open('nombre_del_archivo', 'modo') as archivo:
 # Operaciones con el archivo
 contenido = archivo.read()
```
```

- `open('nombre_del_archivo', 'modo')`: Abre el archivo especificado en el modo indicado (`'r'` para lectura, `'w'` para escritura, `'a'` para agregar, etc.).
 - `as archivo`: Asigna el objeto del archivo a la variable `archivo` para que pueda ser utilizado dentro del bloque.
 - **Bloque indentado**: Contiene las operaciones que se realizarán con el archivo.
-

Ventajas del Uso de `with`

1. **Gestión Automática de Recursos:** El archivo se cierra automáticamente cuando se sale del bloque ``with``, eliminando la necesidad de llamar manualmente a ``archivo.close()``.
 2. **Código más Limpio y Legible:** Reduce la cantidad de código necesario para manejar archivos, haciendo que el código sea más fácil de leer y mantener.
 3. **Manejo de Excepciones:** Asegura que el archivo se cierre correctamente incluso si ocurre una excepción durante la lectura o escritura.
-

Ejemplos de Uso de ``with``

Leer un Archivo

El siguiente ejemplo muestra cómo leer el contenido de un archivo utilizando ``with``:

```
```python

with open('ejemplo.txt', 'r') as archivo:
 contenido = archivo.read()
 print(contenido)

```
```

En este ejemplo, el archivo ``ejemplo.txt`` se abre en modo de lectura (``'r'``). El contenido del archivo se lee y se almacena en la variable ``contenido``, que luego se imprime. Al salir del bloque ``with``, el archivo se cierra automáticamente.

Escribir en un Archivo

El siguiente ejemplo muestra cómo escribir datos en un archivo utilizando ``with``:

```
```python

with open('salida.txt', 'w') as archivo:
 archivo.write('Hola, mundo\n')
 archivo.write('Este es un archivo de ejemplo.\n')

```
```

En este caso, el archivo ``salida.txt`` se abre en modo de escritura (``'w'``). Se escriben dos líneas de texto en el archivo. Al finalizar el bloque ``with``, el archivo se cierra automáticamente.

Leer un Archivo Línea por Línea

El siguiente ejemplo muestra cómo leer un archivo línea por línea utilizando `with`:

```
```python  

with open('ejemplo.txt', 'r') as archivo:
 for linea in archivo:
 print(linea, end="")

```
```

En este ejemplo, el archivo `ejemplo.txt` se abre en modo de lectura y se itera sobre cada línea del archivo, imprimiéndola. La función `end=""` en `print` evita la adición de una nueva línea adicional, ya que cada línea leída ya incluye un carácter de nueva línea.

Conclusión

El uso de la función `with` en Python es una práctica recomendada para el manejo de archivos. Asegura que los archivos se cierran correctamente, facilita la gestión de recursos y mejora la legibilidad del código. Incorporar `with` en tus programas te ayudará a evitar errores comunes y a escribir código más limpio y robusto.

Material complementario:

Recursos complementarios

Google Colab

- [Ejercicios de repaso](#)
- [Ejemplo clase en vivo](#)

Tutoriales

- [Tutorial Git](#)